Generalizing a function

One way to make the `profit` function more generally useful is by making `overhead`, `costPrice`, and `numberSold` arguments:

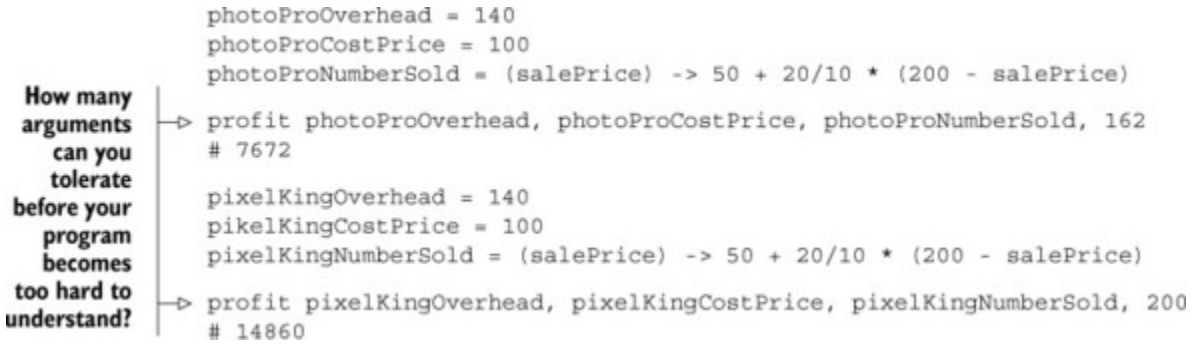
```
profit = (overhead, costPrice, numberSold, salePrice) ->
  revenue = (salePrice) ->
    (numberSold salePrice) * salePrice
  cost = (salePrice) ->
    overhead + (numberSold salePrice) * costPrice

  (revenue salePrice) – (cost salePrice)
```

If the sale price is 100, the overhead is 10, the cost price is 40, and 10 products are sold, then the profit is 590:

```
tenSold = -> 10
profit 10, 40, tenSold, 100
# 590
```

The number sold for a product depends on other values such as the sale price. This is apparent when you use this new function for a photoPro camera and a pixelKing camera.



Flexibility has come at a price. The function now has four arguments, which is unwieldy and a little confusing. Don’t wake up one day to realize that all of your functions take nine arguments. Instead of a program made up of functions with more arguments than you can easily comprehend, use another little bit of function glue—the partial application—and return a function.

Partial application